

UNIVERSIDAD DEL VALLE
SEDE TULUÁ
INGENIERÍA DE SISTEMAS

DOCUMENTO DE ARQUITECTURA
DEL SISTEMA

Proyecto: TaskFlow

		Integrantes:	Jota Emilio López Ramírez — 2259394 Esmeralda Rivas Guzmán — 2259580
Versión	1.0	Profesor:	Kevin Steven Victoria Ospina
Fecha	22 de abril de 2026	Cliente:	ProTask Solutions
Estado	Aprobado		

Índice

1. Visión general de la arquitectura	2
1.1. Capas del sistema	2
1.2. Diagrama de capas	3
2. Stack tecnológico	4
2.1. Versiones fijadas	4
3. Modelo de comunicación entre capas	5
3.1. Protocolo y formato	5
3.2. Códigos de respuesta	5
4. Flujo de autenticación y control de acceso	5
4.1. Flujo completo de login	5
4.2. Roles del sistema	6
4.3. Middleware de autorización en Express	6
5. Estructura de carpetas del repositorio	6
6. Despliegue en Docker Compose	7
6.1. Variables de entorno requeridas	7

Visión general de la arquitectura

TaskFlow sigue una arquitectura de **tres capas** desacopladas, comunicadas a través de una API REST con autenticación basada en JWT. Las tres capas se ejecutan como servicios independientes dentro de un entorno Docker Compose.

Capas del sistema

Capa	Responsabilidad	Tecnología
Cliente	Renderiza vistas, gestiona el estado de la sesión y consume la API del servidor.	React 18 + Vite
Servidor	Expone endpoints REST, valida peticiones, aplica reglas de autorización por rol y persiste datos.	Node.js 20 + Express
Persistencia	Base de datos relacional con todas las entidades del sistema.	PostgreSQL 16

Cuadro 1: Capas de la arquitectura TaskFlow

Diagrama de capas

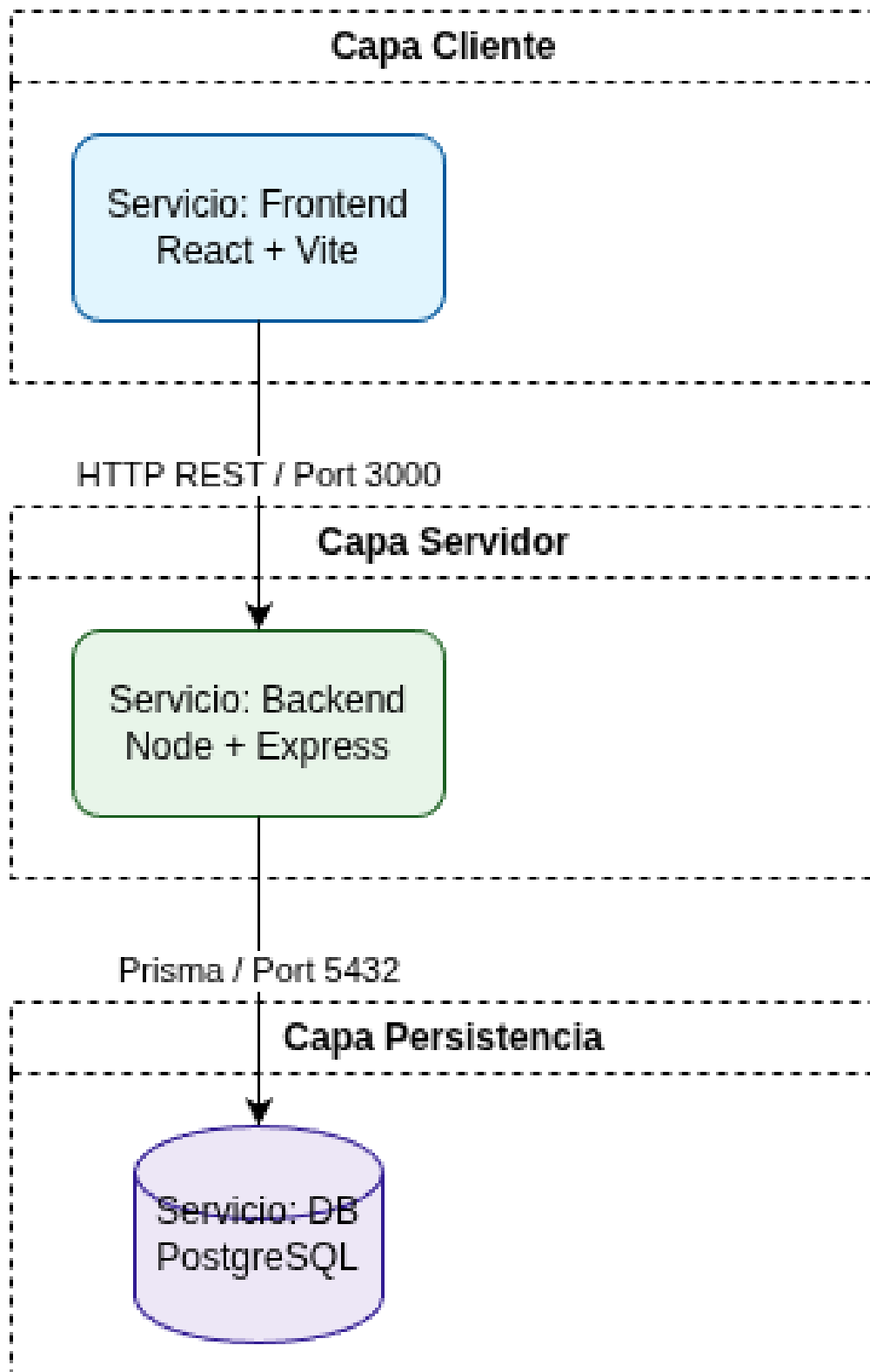


Figura 1: Arquitectura de tres capas de TaskFlow desplegada en Docker Compose

Stack tecnológico

Versiones fijadas

Componente	Tecnología	Justificación	Versión
Frontend	React	Librería de UI con ecosistema maduro; componentes reactivos ideales para el tablero Kanban.	18.x
Build tool	Vite	Compilación y HMR significativamente más rápidos que Create React App.	5.x
Estilos	TailwindCSS	Clases utilitarias; elimina la necesidad de archivos CSS separados.	3.x
Gráficas	Recharts	Librería declarativa de gráficas para React.	2.x
Backend	Node.js	Entorno asíncrono; coherencia de lenguaje con el frontend.	20 LTS
Framework	Express	Framework minimalista y bien documentado.	4.x
ORM	Prisma	Migraciones automáticas, cliente tipado.	5.x
Validación	Zod	Validación de esquemas con inferencia de tipos.	3.x
Auth	JWT + bcrypt	Estándar para APIs REST sin estado.	jsonwebtoken 9.x
Archivos	multer	Middleware para carga de archivos multipart.	1.x
Base de datos	PostgreSQL	RDBMS robusto con soporte completo para transacciones.	16
Contenedores	Docker + Compose	Entorno reproducible y portable.	Docker 25.x
Control de versiones	Git + GitHub	Repositorio remoto con ramas por feature.	—

Cuadro 2: Stack tecnológico con versiones fijadas

Modelo de comunicación entre capas

Protocolo y formato

- **Protocolo:** HTTP/1.1 sobre TCP.
- **Formato de datos:** JSON en el cuerpo de todas las solicitudes y respuestas.
- **Base URL de la API:** `http://localhost:3000/api` (desarrollo) / `http://backend:3000/api` (red Docker interna).
- **Autenticación:** Token JWT enviado en el header `Authorization: Bearer <token>`.

Códigos de respuesta

Código	Significado en TaskFlow
200 OK	Operación exitosa con cuerpo de respuesta
201 Created	Recurso creado correctamente
204 No Content	Eliminación exitosa
400 Bad Request	Error de validación (respuesta incluye detalles)
401 Unauthorized	Token ausente, expirado o inválido
403 Forbidden	Token válido pero el rol no tiene permiso
404 Not Found	El recurso solicitado no existe
409 Conflict	Recurso duplicado (correo ya registrado)
500 Internal Server Error	Error no controlado en el servidor

Cuadro 3: Códigos HTTP usados en la API

Flujo de autenticación y control de acceso

Flujo completo de login

1. El usuario ingresa correo y contraseña en el formulario de login (frontend).
2. El frontend envía `POST /api/auth/login` con `{email, password}`.
3. El servidor consulta la base de datos por el correo recibido.
4. Si no existe el usuario: responde `401` con mensaje genérico.
5. Si existe: compara la contraseña con el hash bcrypt almacenado.
6. Si no coincide: responde `401` con mensaje genérico.
7. Si coincide: genera un JWT firmado con el secreto del servidor, incluyendo `{userId, email, role}` en el payload, con expiración de 8 horas.

8. El frontend almacena el JWT en el **AuthContext** (memoria de React).
9. Todas las solicitudes posteriores incluyen el token en el header **Authorization**.

Roles del sistema

Rol	Permisos principales
ADMIN	Acceso completo. Gestión de usuarios, proyectos, tareas y visualizaciones.
LEADER	Gestión de proyectos propios. Crear, editar y eliminar tareas. Asignar ejecutores. Ver dashboards.
EXECUTOR	Acceso a proyectos asignados. Mover tareas entre fases. Registrar tiempo. Agregar comentarios y adjuntos.

Cuadro 4: Roles y permisos en TaskFlow

Middleware de autorización en Express

1. **authenticate**: verifica la firma y vigencia del JWT. Si falla, responde 401.
2. **authorize(roles)**: verifica que el rol del token esté en la lista de roles permitidos para esa ruta. Si falla, responde 403.

Estructura de carpetas del repositorio

```
taskflow/
  docker-compose.yml      # Orquestación de los tres servicios
  .env.example            # Variables de entorno (plantilla)
  .gitignore              # Node, .env, dist, node_modules
  README.md               # Instrucciones de instalacion y uso

docs/                     # Documentacion del proyecto
  arquitectura/
    arquitectura-sistema.tex
    diagrama-capas.png
    ...

backend/                  # Servicio Node.js + Express + Prisma
  Dockerfile
  package.json
  src/
    app.js
```

```

    routes/
    controllers/
    middlewares/
    ...
  prisma/
    schema.prisma
    migrations/

frontend/                                # Servicio React + Vite + Tailwind
  Dockerfile
  package.json
  src/
  ...

```

Despliegue en Docker Compose

Servicio	Imagen base	Puerto	Notas
db	postgres:16-alpine	5432	Volumen pgdata para persistencia.
backend	Build local	3000	Depende de db con healthcheck.
frontend	Build local	5173	Depende de backend. HMR de Vite habilitado.

Cuadro 5: Servicios en docker-compose.yml

Variables de entorno requeridas

```

# -- Base de datos -----
POSTGRES_USER=
POSTGRES_PASSWORD=
POSTGRES_DB=

# -- Backend -----
DATABASE_URL=
JWT_SECRET=
JWT_EXPIRES_IN=
NODE_ENV=
PORT=3000

# -- Frontend -----
VITE_API_URL=

```